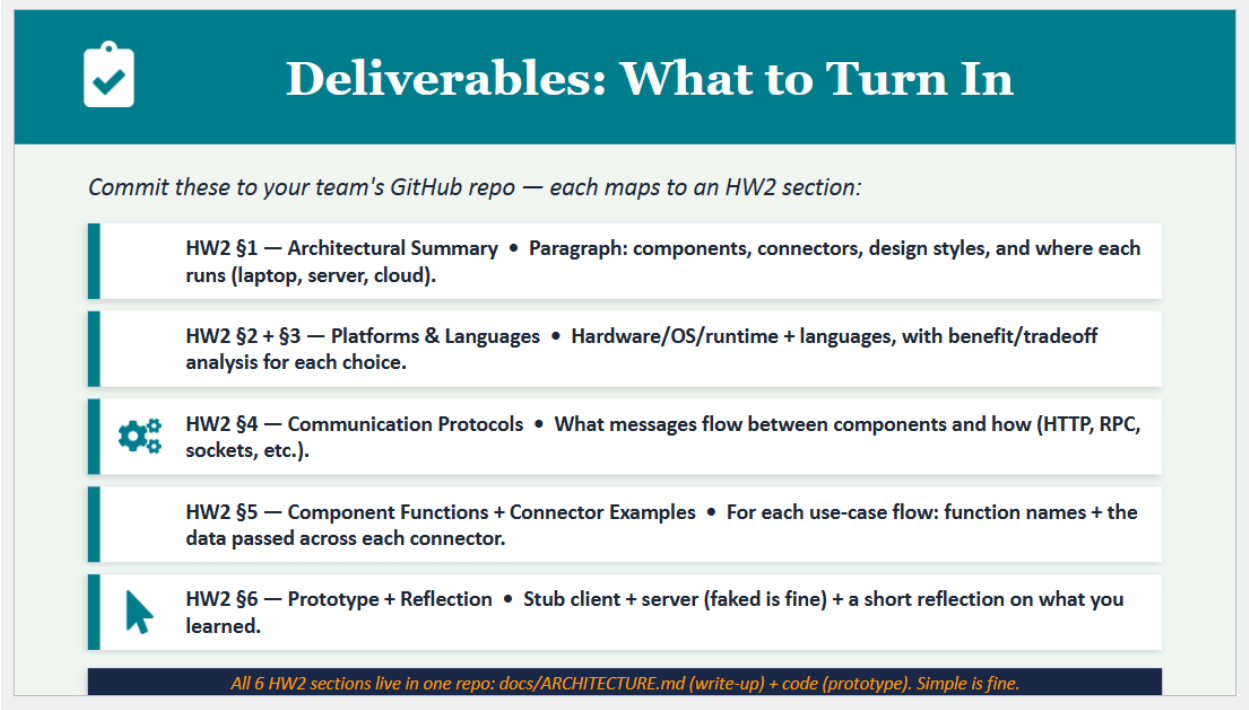


Overall Architectural Summary

With respect to your project requirements as documented by your specification document (i.e., HW1), describe your envisioned architecture in terms of its components and connectors, its design style(s), and how everything fits together. For each component, describe the key functionalities. For each connector describe the data that are communicated. For each design style, describe how they are used and the key benefits that that style brings to your project. Describe where each component will run (e.g., on a mobile device, on a server, in an AWS cloud).

The architecture for our app is a mobile application, built around client-server communication using React Native/Expo, a Node.js API server, an SQL database, and external services such as Google Maps and ID Verification service. The mobile app will run on Android devices, and handles user registration, ID uploads, user interface, profile updates, location changes, user-to-user messages, activity joining and privacy setting updates. The mobile app will communicate with our server with HTTP requests using JSON bodies. The Node.js server will act as the primary controller in handling the logic and updates on login, registration, activity joining, ID verification, profile change, filter processing, etc. Data such as user information, recurring activity lists, messages, blocks/privacy settings will be stored in our SQL database through CRUD operations. The Google Maps SDK/API will receive information about the users' locations, which will be communicated through our server API before being relayed to another user, for encryption. The ID verification services will not permanently store any user ID

information, but will return whether if the user could be verified for registration. Overall, our app design uses a layered architecture, where each of the interconnected component of the system has a different responsibility.

Platforms: *Specify on what platforms (e.g., hardware categories, operating systems, runtime frameworks) the different parts of your system will run. Provide an analysis of the benefits and trade offs of each of those choices.*

The platforms that we using are:

1. Android
    a. Coding Language: Expo React Native
    b. Part: Mobile App
    c. This is our main platform, because all types of platforms can test an android environment.
        i. Benefits: The benefits of using react is that the platform will respond quickly, and can conform with IOS applications later.
        ii. Tradeoffs: For configuration, we cannot just use react, we will have to use APIs which cause more components that need to be hidden by abstraction.
2. Linux
    a. Coding Language PaaS + Node.js
    b. Part: Server
        i. Benefits: Can use JSON when it comes to connecting APIs which makes it easier to load data in a set formatted way. Most of us are learning how to iterate data of JSON from APIs in ICS H32, so we are most familiar with it.
        ii. The CPU is doing a lot of the work, so users' phones/computers could heat up, the application can be slower, etc.
3. Vendor SDK and HTTPS API
    a. Part: Maps
        i. Benefits: This is a reliable way to receive geolocation and have an accurate map.
        ii. Expensive, privacy concerns, and some parts of this can only be accessed by android, so if we wanted to expand to IOS it would be harder.
4. SQL
    a. Part: Data
        i. Benefits: Can query easily and it will not bug out over time.
        ii. Cost: Backups can be difficult to perform.

Programming Languages: *Choose the programming languages that will be used by each part of your system. Provide an analysis of the benefits and trade offs of each of those choices.*

The programming languages that we are choosing to use are:

1. React Native and React in combination
    a. The role of react will be for the UI/UX design
        i. Benefits: React supports reusability. The type of user interface this would be is declarative, meaning we get to focus on what it should look like before we focus on the how. This is something that we learned in Informatics 43.
        ii. Tradeoffs: Connecting modules in React Native is harder than other languages, so it might be tedious. Additionally, code may need to be edited a good amount and that could cause bugs.
2. Node.js
    a. The role of node is to use it as an API, specifically HTTP. It would be public.
        i. Benefits: React and node would be used for the frontend, giving more flexibility.
        ii. Tradeoffs: More errors, and need to ensure there are authentication/privacy protocols that are covered when receiving data from the API.
3. Python
    a. The role of this is to give us more flexibility with services. For example we have more geolocation helpers.
        i. Benefit: Very large library for geolocation as well as helps with asynchronous activity.
        ii. Clashes with nodes because they both have different interface agreements as well as how they run code.

Communication Protocols: *Describe what messages need to be sent and/or requested from each component or part of the system to other components or parts of the system. Define how those messages will be sent (e.g., HTTP requests, remote procedure calls (RPC), TCP sockets, UDP sockets).*

1. **From the Mobile App to the API Server**
    a. Will use HTTPS with JSON request bodies. This will help with protecting the information that is sent as HTTPS allows us to encrypt messages between parts of the system allowing for greater security. This is extremely valuable especially considering that our app will rely on users giving the app their ID and passwords to verify their identity. JSON works perfectly well with our other parts which will be using React Native and Node.js , since it's compatible and easy to read.
        i. The type of information that will be sent will be:
            1. Login and registration request: Username, password hash, email, and the ID image

2. Location updates: The users current location sent periodically via their longitude and latitude
3. Chat Messages: Sender ID, Recipient ID and message content
4. Profile changes: Bio text, public/private setting, and the filter settings
5. Activity request: Join, create, view local group activity, includes the location, name, time

2. From API Server to SQL Database

a. The pattern in which we will send and receive data from our SQL database is via CRUD/SQL commands, which stand for (Create, Read, Update and Delete). Every entity in our app would map to a table such as a table for users, events and chats and the server would interact with the tables through SQL commands.

i. Process of using each command:

1. Create: Initially there would be a table created for the entity with the first couple of people that register for the app. When new users register then a new row would be added with their information on it in the Users table.
2. Read: An example of this implementation would be when a user opens up their chat tab, then when they do the API server would query the database for that user's chat history and that is how it would populate the chat.
3. Update: When a user edits their bio, their public/private setting, or changes their filter then the row in the users table would be updated for the corresponding individual.
4. Delete: When a user deletes their account, then we would simply delete their row on all of the tables that they are associated with.

3. From the API Server to the maps SDK (Google Maps)

a. The server uses HTTPS to communicate with the third-party maps provider. The process is as follows where the server sends a combination of the longitude and latitude of the user's current location, and receives the map tile data or geocoded location info from the SDK. The mobile app is able to render the map on the app by communicating with the SDK as well but mainly we try to send location info for other users within the current users area through our server API to the SDK for that extra encryption protection.

4. From the API Server to the ID verification service

a. As the user is registering for their account and they have to verify their identity then they would upload a photo of a government-issued ID. This photo is then sent by the server to a third party ID verification service. This message is through most often the form of HTTPS POST, which is what we will be using. Once the server receives the message then it either verifies the users identity by verifying

that their ID and name match or it sends a message back of denial. Depending on the acceptance or denial, the server allows the user to continue or blocks them from registering.

Examples of Component Functions and Connector Communications: *For each of the use case flows that you described in your specifications document, give an example of the functions (on a component) that handle each part of the overall computational workflow, and examples of the data that is communicated in each step (across a connector). (Anirudh/Daniel)*

1. *Age Range*
    a. *Basic flow:*
        i. *User opens the settings app: backend prompts user_open_settings() -> frontend displays current settings*
        ii. *App displays current range values: Frontend.display_range_data()->Backend sends an HTTPS GET /api/settings/range to server and gets back { currentRange: number }*
        iii. *User toggles with range data bar or types in data: Settings Frontend.change_range_values() -> { new_Range: number } is saved by the Backend and set to the server through an HTTP PUT request*
        iv. *Database updated with new values:Backend.update_values()->SQL Database updates range values with a UPDATE command.*
        v. *SQL Database confirms with the server: Database returns the row to the server -> Backend responds with an HTTP Ok 200 confirmation statement.*
        vi. *Map refreshes to the changes: Frontend.filter_users_from_range() -> Backend sends an HTTP request to the google maps API to get information(User_id, range: number) about which users are in the area.*
    b. *Alternative flow:*
    (Repeat of the first two steps in the Basic flow)
        i. *Users toggles the bar or increments out of range: Frontend attempts the change_range_values() -> Backend correctly points out the issues using validateRange() returning the reason:('increment constraint')*
        ii. *App attempts to resolve the issue by capping the value: Frontend displays the highest possible value with display_highest_range() function-> {range: number} saved locally by the app*
        iii. *App saves the range: Backend calls the saveRange() function to save the value and sends out a statement to the server -> An HTTP PUT request is sent to the server with information on {user_id, range} with range being the max or min range.*

iv. *Database accepts and stores the value: The server stores the value on the SQL database -> UPDATE [database_name] SET range_km command sent*
v. *Backend confirms and more steps occur like stated in the control flow.*

c. *Exception Flow:*
i. *User opens the settings section on the application: Frontend uses the display_info() to show the user options on what they can use-> user credentials already established.*
ii. *User inserts themselves into the search tool and searches for a non-existent entity range: Frontend displays common searches and adjusts the toolbar display with tool_format() function -> Backend stores the text that is written by the user with the store_search_bar_text()*
iii. *App searches for information about the tool: Backend sends out a request to the server about information about that certain tool written -> HTTP GET /api/settings/info/range*
iv. *Server tries to get information about the tool from the database: Server searches for information on functionality -> SELECT range FROM info*
v. *Database gives faulty information: Database returns an empty list -> []*
vi. *Server informs backend of lack of information : HTTP ERROR -> []-backend returns a blank statement*
vii. *Users see on the app that information on the topic cannot be found: Frontend calls showEmptyState() function -> Displays message "No information on requested service could be found."*

2. *Real Names:*
a. *Basic Flow:*
i. *User starts the registration process: registration_window() called on frontend: state set to name entry*
ii. *App gains the information about the people's names: Frontend calls gathernameinfo() -> {first_name, last_name} is stored locally at first*
iii. *Submit button is pressed after unsuspicious name is passed: submit() is called from the frontend-> the {first_name, last_name} is stored globally and passed over to the backend*
iv. *App starts verification of identity: validate_name() is called by the backend sending out an HTTP request to the server -> HTTP POST /api/profile/name {user_id, display_name}*
v. *Server searches the database: verify_name() is called -> SELECT {first_name, last_name} FROM users sent to the SQL database is called and return true as an empty list is found so {valid:true}*
vi. *Server saves the name to the database: save_name() is called -> UPDATE users SET name = {name} WHERE id = ? sent to database*

vii. *Server confirms addition of the name: Server confirms that it was able to add the name -> HTTP 200 CREATED*
viii. *Backend returns that the name was valid: validate_name returns truthy information -> {suspicious: False, valid: True}*
ix. *App advances to the next step: Frontend calls advance_step() -> state no longer is in the name section*

b. *Alternative Flow:*
   i. *User submits a suspicious name: Frontend calls submit() -> the {first_name, last_name} is stored globally and passed over to the backend; {first_name = "stupid", last_name = "app"}*
   ii. *Backend detects that the name is suspicious: validate_name() is called -> {suspicious: true, valid: False}*
   iii. *App displays information about why they caught an error: Frontend calls displaysusnamevalue() -> {message: Name seems suspicious, attempts_left:4}*
   iv. *User re-submits a valid name: Frontend calls submit() again -> a valid {first_name, last_name} is stored globally and passed over to the backend*
   v. *Name is verified : validate_name is called (steps 4-8 in basic flow are repeated) -> {suspicious: False, valid: True}*
   vi. *App advances to the next step: Frontend calls advance_step() -> state no longer is in the name section*

c. *Exception Flow:*
   i. *User submits a suspicious name: Frontend calls submit() -> the {first_name, last_name} is stored globally and passed over to the backend; {first_name = "stupid", last_name = "app"}*
   ii. *Backend detects that the name is suspicious: validate_name() is called -> {suspicious: true, valid: False}*
   iii. *App displays information about why they caught an error: Frontend calls displaysusnamevalue() -> {message: Name seems suspicious, attempts_left:0}*
   iv. *Registration becomes locked for the user: Frontend calls the lockRegistrationField() -> state = {locked, flag_account = True, message = "Too many suspicious tries."} passed to backend as well*
   v. *Information about the lockage is displayed: displayLockageReason() is called by the frontend - > {message: "You took too many tries."}*
   vi. *Backed flags the account: flagAccount(user) is called by the backend sending out a request to the server -> HTTP POST /api/flag called {user_id, reason} passed*

vii. *Database marks the account -> punish_account() is called by the server prompting the SQL database to save the account info-> INSERT INTO flag {user_id, reason}*

3. *Private & Public Settings:*
    a. *Basic Flow:*
        i. *User opens settings: Frontend calls display_settings() -> state = {in_setting}*
        ii. *User selects on the privacy option: Frontend calls the display_privacy() option -> updates state = {in_setting, privacy}*
        iii. *Backend loads the information about the current privacy state: load_privacy_state() calls the server -> HTTPS GET /api/settings/privacy sent to the server*
        iv. *Server sends response about the setting -> {visibility: public}*
        v. *User changes the visibility: Frontend calls the toggle_visibility_bar() -> Bar toggles to private and visibility is set to private locally*
        vi. *Backend saves the visibility to the system: saveVisibilitySetting() is called by the backend -> HTTPS PUT api/settings/privacy {"visibility": "private"} sent to server*
        vii. *Server responds with a successful change: successful_change() called -> HTTP 200 OK*
    b. *Alternative Flow:*
        i. *User opens settings: Frontend calls display_settings() -> state = {in_setting}*
        ii. *User selects on the privacy option: Frontend calls the display_privacy() option -> updates state = {in_setting, privacy}*
        iii. *Backend loads the information about the current privacy state: load_privacy_state() calls the server -> HTTPS GET /api/settings/privacy sent to the server*
        iv. *Server sends response about the setting -> {visibility: public}*
        v. *User changes the visibility multiple times quickly: Frontend calls the toggle_visibility_bar() multiple times in a small time frame-> Bar toggles to private and public and back and forth but isn't able to keep up*
        vi. *Backend attempts to save the visibility to the system: saveVisibilitySetting() is called by the backend -> HTTPS PUT api/settings/privacy {"visibility": "private/public"} sent to server however multi-states are sent at once*
        vii. *Server responds with an error: failed_change() called -> HTTP 500 ERROR*

viii. *App displays an error message: Frontend calls display_too_much_user_input() -> {reason: "Server error", message: "Please don't toggle the bar too much."}*
ix. *User listens to request: Frontend calls the toggle_visibility_bar() once-> Bar toggles to private and visibility is set to private locally*
x. *Backend saves the visibility to the system: saveVisibilitySetting() is called by the backend -> HTTPS PUT api/settings/privacy {"visibility": "private"} sent to server*
xi. *Server responds with a successful change: successful_change() called -> HTTP 200 OK*

c. *Exception Flow:*
i. *User opens settings: Frontend calls display_settings() -> state = {in_setting}*
ii. *User selects on the search: Frontend calls the search() option -> updates state = {in_setting, search}*
iii. *User searches for the privacy setting: Frontend calls the function get_info("privacy")*
iv. *Backend makes a request to the server to find the information needed: get_info_about_setting("privacy") -> HTTPS GET /api/settings/privacy sent to the server*
v. *Server cannot locate the setting: Server send back information_error() -> HTTP Error 404(Not Found)*
vi. *App displays error message: Frontend displays calls display_feature_not_found() -> {error: "feature not found"}*

4. *Chat Feature:*
a. *Basic Flow:*
i. *User opens a chat tab: Frontend displays the chat tabs call display_chat(): {state: "chat"} -> passed to backend*
ii. *Backend calls the server: Backend calls connect_to_others() to connect to the server -> HTTP GET api/connections sent to the server*
iii. *Server responds listing users connected: Server calls load_users(): [User1, User2, User3]*
iv. *User selects which user that they want to chat to: Frontend calls selectUser()-> user: "{selected}" passed to backend*
v. *App initiates the process to load messages through a server request: Backend calls loadMessages("{selected}") -> HTTP GET /api/messages/selected send to the server*
vi. *Server gains the conversation id from the database: get_converstation() called by server -> SELECT * FROM messages WHERE selected = ? Values=(Selected)*

vii. *App displays conversation and user types in message: show_messages() and typed_message() called by frontend -> {loaded = True, message_submitted = True}*
viii. *Application posts the message to the backend: postMessage(selected, message) -> HTTPS POST /api/messages {, selected, text} sent to server*
ix. *Server writes the message into the database: SaveMessage() called in server -> SQL INSERT INTO messages(user, selected,text) sent to SQL database*
x. *Message is pushed from local server to websocket for receiver to get it: WebSocket event {msgId, text, senderId, }pushed to recipient's connected client*
xi. *Recipient receives message on the other end: recieveMessage() called on their local app server: Message inserted into conversation database*

b. *Alternative Flow:*
   i. *User sends a message: Frontend calls onSendMessage() -> {text: [message_content]}*
   ii. *Backend of the app pushes to message: pushMessage() -> HTTP Post /api/message sent to the server*
   iii. *Connection is away so the reply is not immediate.: status() is called -> ["away"]*
   iv. *App schedules a notification for a reply: Frontend calls scheduleNotification() -> HTTP POST /api/notifications/schedule*
   v. *Friend responds; Server receives message and sends via WebSocket: Websocket event {msg, text_content}*

c. *Exception Flow:*
   i. *App loads message: Frontend calls loadMessages(text).-> HTTP GET /api/messages/connection called*
   ii. *App checks if any of the content that was sent was inappropriate: detect_disturbing_content() will be called by the server -> {flag:True}*
   iii. *User is given an option to block and report on the application: block_and_report() called by the frontend -> mode: {block:{true/false}*
   iv. *If the user chooses to block the user the backend sends the request to the server: user_block(person) -> HTTPS POST /api/reports with { reporterId, offenderId, reason }*
   v. *Serves updates that database to ensure that the person is blocked -> SQL INSERT INTO blocks(user_id, reason)*
   vi. *Server sends back confirmation that the user was successfully blocked -> HTTP 200 OK*

5. *Publicly Viewed Bios*
    a. *Basic Flow*
        i. *User opens Profile: Frontend calls display_profile → display_profile is sent to Backend and Backend holds the data about which profile is to be displayed*
        ii. *Backend calls the server: Backend calls load_profile() → HTTPS GET /api/profile/me → server returns {bio: "...", user_id: "123"}*
        iii. *User clicks Edit Bio: Frontend calls edit_bio() → Backend receives and updates state = {edit_move_active}*
        iv. *Save Bio: Frontend calls save_bio() → Backend calls analyze_language() and checks {new_bio: "..."} for any inappropriate language → HTTPS Post /api/profile/update {new_bio: ".."}*
        v. *Store in Server: server calls update_bio() → information is stored in server, HTTP 200 OK → Frontend is properly updated and displaying correctly*
    b. *Alternative Flow*
        i. *User opens Profile: Frontend calls display_profile → display_profile is sent to Backend and Backend holds the data about which profile is to be displayed*
        ii. *Backend calls the server: Backend calls load_profile() → HTTPS GET /api/profile/me → server returns {bio: "...", user_id: "123"}*
        iii. *User clicks Edit Bio: Frontend calls edit_bio() → Backend receives and updates state = {edit_move_active}*
        iv. *User clicks Bio Prompts: Frontend calls display_prompts() → HTTPS GET api/templates/prompts*
        v. *User selects Prompt: Frontend calls load_prompt → Backend receives and state = {prompt_active}*
        vi. *Save Bio: Frontend calls save_bio() → Backend calls analyze_language() and checks {new_bio: "..."} for any inappropriate language → HTTPS Post /api/profile/update {new_bio: ".."}*
        vii. *Store in Server: server calls update_bio() → information is stored in server, HTTP 200 OK → Frontend is properly updated and displaying correctly*
    c. *Exceptional Flow:*
        i. *User opens Profile: Frontend calls display_profile() → display_profile() is sent to Backend and Backend holds the data about which profile is to be displayed*
        ii. *Backend calls the server: Backend calls load_profile() → HTTPS GET /api/profile/me → server returns {bio: "...", user_id: "123"}*

iii. *User clicks Edit Bio: Frontend calls edit_bio() → Backend receives and updates state = {edit_move_active}*
iv. *Save Bio: Frontend calls save_bio() → Backend calls analyze_language() and checks {new_bio: "..."} for any inappropriate language → HTTPS Post /api/profile/update {new_bio: ".."}*
v. *Language Detected: Backend returns HTTP 400 Bad Request {error: "prohibited language"}*
vi. *Error: Frontend calls display_error("Prohibited language") → an error is displayed telling the user what they need to fix*

6. *Filter (gender, age, etc..)*
   a. *Basic Flow*
      i. *User opens Filter: Frontend calls display_filters() → state = {filter_active}*
      ii. *User adjusts Filters: Frontend calls update_filters(age: "20-25, gender: "female")*
      iii. *User applies Filters: Frontend calls display_filtered() → HTTPS GET /api/search?age_min=20&age_max=25&gender=female*
      iv. *Update Display: server returns all applicable users → Backend calls update_search(info) → Frontend displays all matching users*
   b. *Alternative Flow*
      i. *User opens Filter: Frontend calls display_filters() → state = {filter_active}*
      ii. *User clears Filters: Frontend calls reset_filter() → state={empty_filters}*
      iii. *User refreshes display: Frontend calls display_filtered() → HTTPS GET /api/search*
   c. *Exceptional Flow*
      i. *User opens Filter: Frontend calls display_filters() → state = {filter_active}*
      ii. *User adjusts Filters: Frontend calls update_filters(age: "20-25, gender: "female")*
      iii. *User applies Filters: Frontend calls display_filtered() → HTTPS GET /api/search?age_min=30&age_max=30&gender=female&dist=1*
      iv. *Update Display: Server returns all applicable users → Backend calls update_search(info) → info is found to be None*
      v. *Show Users: Backend calls display_empty() → Frontend displays Text telling the user that there is no user matching criteria as well as a button to widen search criteria*
      vi. *Broaden Search: Frontend calls expand_search() → HTTPS GET /api/search?age_min=25&age_max=35&gender=female&dist=5*

vii. *Display Expanded: Backend calls update_search(info) → Display is updated and contains users*

7. *ID-based registration*
    a. *Basic Flow*
        i. *User registration: Backend calls request_id() → state = {need_id}*
        ii. *Reminder displayed: Backend calls display_reminder() → state = {id_reminder}*
        iii. *User enters ID: Frontend calls upload_image() → HTTPS POST /api/verify/documentation {"image": id-data}*
        iv. *Verification: Backend calls validate_document() → state = {validating_image}*
        v. *Access: Server returns HTTP 201 Created {"verification": True} → state = {registration_done}*
    b. *Alternative Flow*
        i. *User registration: Backend calls request_id() → state = {need_id}*
        ii. *Reminder displayed: Backend calls display_reminder() → state = {id_reminder}*
        iii. *User enters ID: Frontend calls upload_image() → HTTPS POST /api/verify/documentation {"image": id-data}*
        iv. *Verification: Backend calls validate_document() → state = {validating_image}*
        v. *Issue: Server sends HTTP 422 Unprocessable Content {code: "image blurry"} → Backend calls request_upload() → The user is told that their image is faulty and requested to reupload their ID.*
        vi. *Fix Image: Backend calls id_guide() → state = {fix_id} → frontend is updated to display a message guiding the user on how to properly photograph their ID*
        vii. *Reupload: user enters ID → Frontend calls upload_image() → HTTPS POST /api/verify/documentation("image": id-data} → Backend calls validate_document() → state = {validating_image}*
        viii. *Access: Server returns HTTP 201 Created {"verification": True} → state = {registration_done}*
    c. *Exceptional Flow*
        i. *User registration: Backend calls request_id() → state = {need_id}*
        ii. *Reminder displayed: Backend calls display_reminder() → state = {id_reminder}*
        iii. *User enters ID: Frontend calls upload_image() → HTTPS POST /api/verify/documentation {"image": id-data}*

iv. *Verification: Backend calls validate_document() → state = {validating_image}*
v. *Issue: Server sends HTTP 422 Unprocessable Content {code: "image blurry"} → Backend calls request_upload() → The user is told that their image is faulty and requested to reupload their ID*
vi. *Timeout: User repeats this process → Server calls timeout() → state = {timedout} → Backend calls timeout_screen() which lets the user know the necessity of ID and gives contact information to the user if they have concerns*
vii. *Flag Account: Backend calls flag_account(user) → user's id is placed in a list to be scrutinized*

8. *Different tab for recurring group activities*
    a. *Basic Flow*
        i. *User clicks Recurring activities: Frontend calls display_activities() → HTTPS GET /api/activities/recurring*
        ii. *User selects activity: Frontend calls load_activity(activity_id)*
        iii. *User joins activity: Frontend calls join_activity(activity_id) → HTTPS POST /api/groups/join {group_id: ".."}*
    b. *Alternative Flow*
        i. *User clicks Recurring activities: Frontend calls display_activities() → HTTPS GET /api/activities/recurring*
        ii. *User selects multiple activities: Frontend calls load_activity(activity_id) for each activity the users clicks on*
        iii. *User joins activity: Frontend calls join_activity(activity_id) → HTTPS POST /api/groups/join {group_id: ".."}*
    c. *Exceptional Flow*
        i. *User clicks Recurring activities: Frontend calls display_activities() → HTTPS GET /api/activities/recurring*
        ii. *User selects activity: Frontend calls load_activity(activity_id)*
        iii. *User joins activity: Frontend calls join_activity(activity_id) → HTTPS POST /api/groups/join {group_id: ".."}*
        iv. *Group full: Server sends HTTP 403 Forbidden {code: "Group Full"}*
        v. *Display error: Backend calls display_error(error) → user is shown a popup detailing the error*
        vi. *Return: Frontend calls display_activities() → display is reverted back to recurring activities*

Prototype Implementation: *Start to create a prototype implementation. We will not be evaluating this prototype at this point, but we are asking you to reflect upon what you have learned from this*

experience. For example, at this stage, you may wish to create a stub client (e.g., no UI, just a function that could be called in a loop) that sends data to a server, which mostly just receives messages and does something with that. These functionalities and communications can be mostly faked for now, just to get some experience with the platforms, technologies, the data that should be sent, etc. Document your experience doing these prototypes, anything that you learned from the process, any challenges that you encountered.

I prompted Cursor to use all of the requirements specification, design decisions made from this week, and other directions from the instructor to come up with a very simple prototype for our app. The prototype uses Node.json files in order to simulate a server and its clients on different terminals. Here are some of the most basic functions that were written in this prototype:

```
async function register() {
  const r = await fetch(`${BASE_URL}/api/register`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      userId: USER_ID,
      displayName: "Demo User",
      email: "demo@example.edu",
    }),
  });
  if (!r.ok && r.status !== 409) {
    const t = await r.text();
    throw new Error(`register failed: ${r.status} ${t}`);
  }
}

async function sendLocation() {
  // Tiny random walk to simulate movement
  lat += (Math.random() - 0.5) * 0.002;
  lon += (Math.random() - 0.5) * 0.002;
  const r = await fetch(`${BASE_URL}/api/location`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ userId: USER_ID, lat, lon }),
  });
  const data = await r.json();
  if (!r.ok) throw new Error(JSON.stringify(data));
  console.log(new Date().toISOString(), "location ok", data.received);
}

async function fetchNearby() {
  const q = new URLSearchParams({ lat: String(lat), lon: String(lon), radiusKm: "30" });
  const r = await fetch(`${BASE_URL}/api/nearby?${q}`);
  const data = await r.json();
  console.log("nearby:", data.count, data.nearby);
}
```

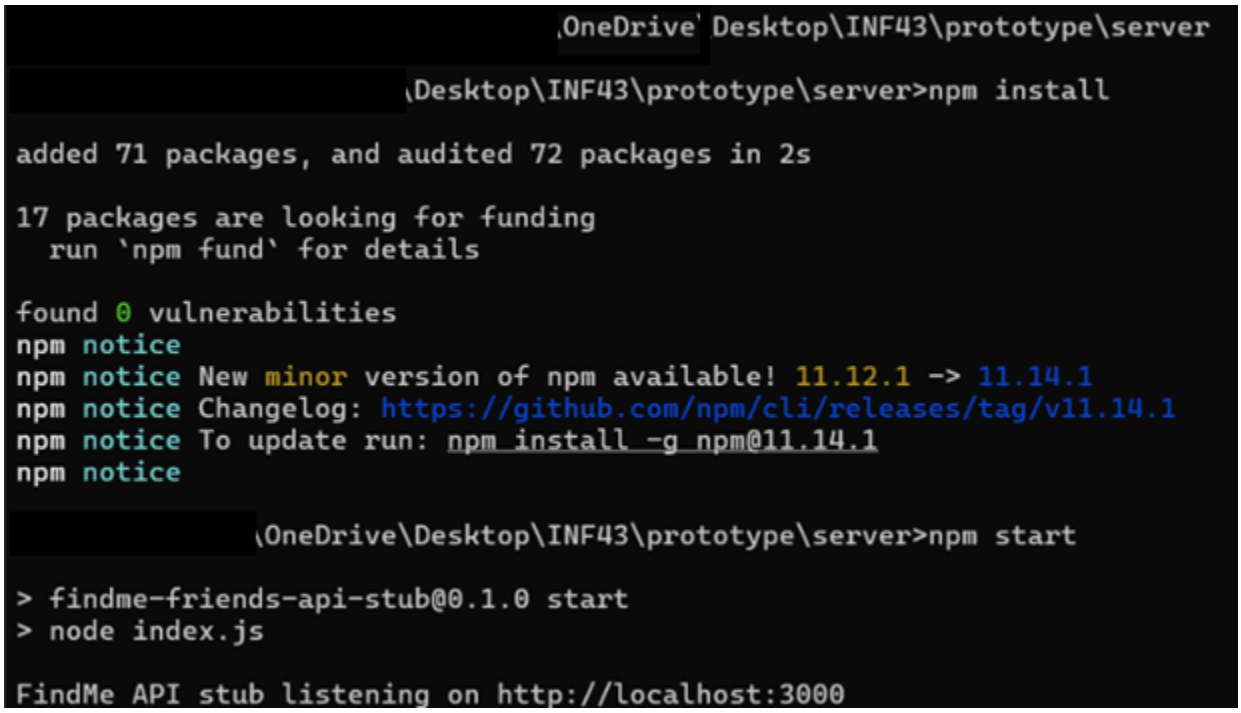
register() simply sends the userId, displayName, and email to the server, and the server takes these information to create an entry in the “users” map.

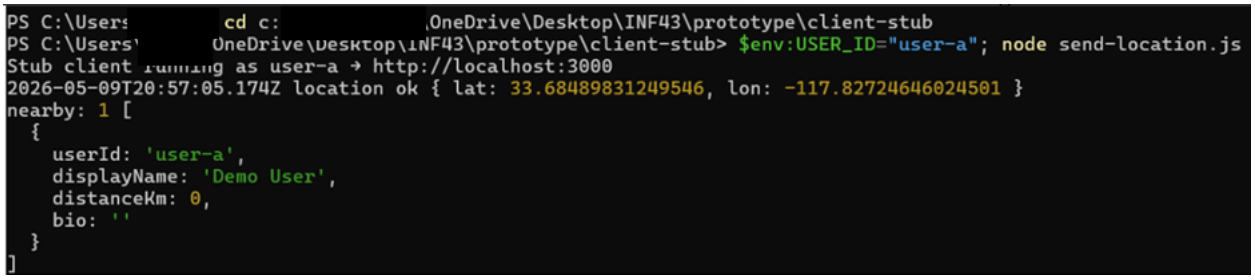
The current implementation of sendLocation() uses random functions to change the user’s location by a small bit to simulate movement, then sends the location variables lat and lon paired with userId to the server. The server then finds that user within its map and overwrites the user’s location.

fetchNearby() builds a query with a single entry of location and radius, and compares the information with each of the users.

A challenge that arises with this design is that each fetchNearby loop will complete a sequential search with the entire user database, which has a O-notation of O(N) and is suboptimal for a realistic, scalable implementation of a mobile app with potentially thousands of users. However, this is a simplified prototype stub that is mostly designed for us to learn the process of building an app. It still does its job, and will have to suffice for our current expectations.

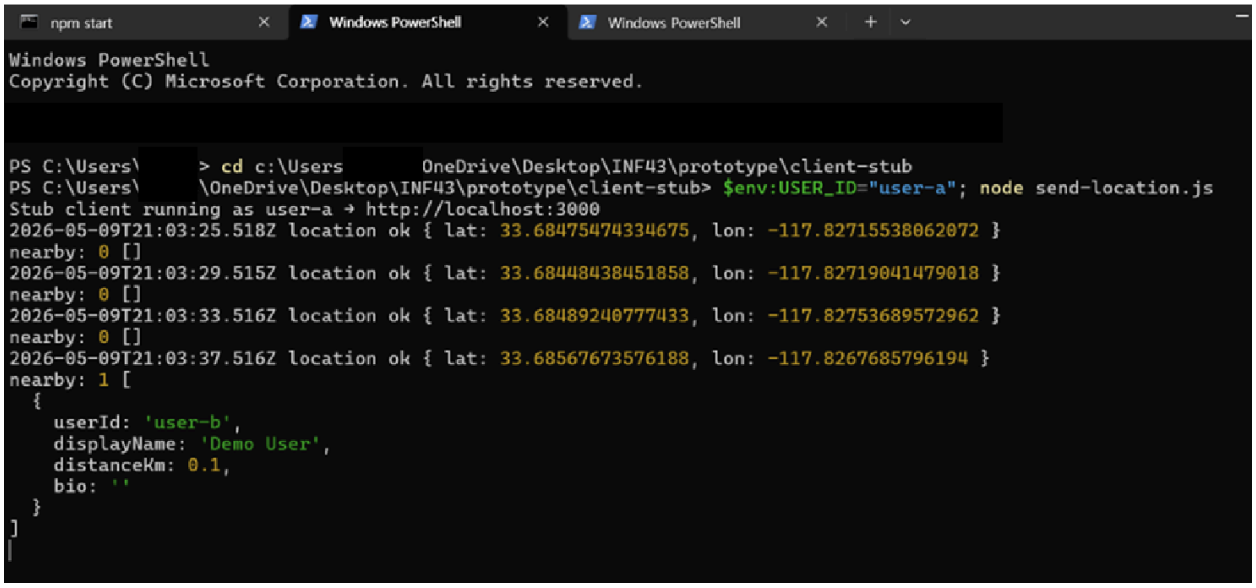
With a few different packages and .js files that include these functions, I have managed to create a local simulation of our server-client communication on my environment.





Another challenge from this prototype was that upon registration of the first user, the server-side comparison didn't actually differentiate the client from other user users. As a result, as seen above, a single registration resulted in a loop falsely reporting that there was a nearby user. I

fixed this issue by assuming every userId was unique, and adding a simple comparison with these ids.



This is a screenshot of one of the client terminals after making these changes. When user-a is inputted into this terminal, and user-a is the only user on our database, the loop fetches noone and reports nearby as 0. After opening another terminal and creating a user-b, the loop responds to this change by fetching the other user.

This is a very basic implementation for the prototype and is missing most of the functionalities that are specified in our requirements specification. However, we were able to familiarize ourselves with the server-client communication mechanisms and the language that we will be using. The issues and design decisions that arose throughout this prototype will be beneficial in how we proceed with our app development.